

NMIN112 Programování 2 2/4 Z+Zk 8 kr.

Pavel Töpfer

Katedra softwaru a výuky informatiky MFF UK

MFF Malostranské nám., 4. patro, pracovna 404

pavel.topfer@mff.cuni.cz

<https://ksvi.mff.cuni.cz/~topfer>

Algoritmizace

- metody řešení úloh na počítači
- algoritmy, datové struktury, programovací techniky

Správnost algoritmu

- konečnost
- parciální korektnost (správné výsledky, když výpočet skončí)

Efektivita algoritmu (složitost algoritmu)

- časová (počet vykonaných operací)
- prostorová (pracovní paměť potřebná na uložení dat)

Učivo

- algoritmus, časová a prostorová složitost
- dělitelnost čísel, Eukleidův algoritmus
- test prvočíselnosti, Eratosthenovo síto
- rozklad čísla na cifry
- aritmetika s vyšší přesností („dlouhá čísla“), polynomy
- Hornerovo schéma, poziční číselné soustavy
- algoritmy vyhledávání v poli (sekvenční, binární, zarážka)
- řazení čísel v poli - přímé metody, heapsort, mergesort, quicksort
- složitost problému třídění
- přihrádkové metody třídění
- vnější třídění
- reprezentace dat v paměti
- zásobník, fronta, halda, slovník
- spojový seznam

Učivo (pokračování)

- rekurze – princip, příklady, efektivita, rekurzivní generování
- binární a obecný strom – reprezentace, průchod, použití
- binární vyhledávací strom, princip vyvažování
- notace aritmetického výrazu – vyhodnocení, převody
- reprezentace grafu
- prohledávání grafu, základní grafové algoritmy
- prohledávání stavového prostoru do hloubky a do šířky
- metody zrychlení backtrackingu – ořezávání, heuristiky
- programování her, algoritmus minimaxu
- metoda Rozděl a panuj
- dynamické programování

Studijní zdroje (přednášky)

Prezentace z přednášek

- po každé přednášce na webu přednášejícího
- materiály (někdy rozšířené a doplněné oproti přednášce)
- ukázkové programy v Pythonu

Studijní zdroje (algoritmy)

Pavel Töpfer: Algoritmy a programovací techniky

Prometheus Praha 1995, 2. vydání 2007

starší text, ale obsahově téměř odpovídá (programové ukázky v Pascalu)
tištěná kniha v knihovnách, nyní k zakoupení pouze jako e-kniha

<https://www.prometheus-eknihy.cz/>

Programátorská encyklopedie KSP

krátké studijní texty k jednotlivým tématům algoritmizace a programování

<http://ksp.mff.cuni.cz/encyklopedie/>

Martin Mareš, Tomáš Valla: Průvodce labyrintem algoritmů

CZ.NIC Praha 2017

text pdf zdarma ke stažení

<http://pruvodce.ucw.cz/>

Studijní zdroje (Python)

Mark Pilgrim: Ponořme se do Python(u) 3

CZ NIC Praha 2011

text pdf zdarma ke stažení

<https://knihy.nic.cz/cs/detail/3/>

odkazy na různé on-line materiály na webu

<https://python.cz/zacatecnici/>

například kurz PyLadies

<https://naucse.python.cz/course/pyladies/>

Zápočet

- uděluje cvičící („praktický“ po souhlasu „teoretického“)
- domácí úkoly z teoretické i z praktické části cvičení
- praktický zápočtový test
- zápočtový program
- další požadavky cvičících (účast na cvičeních, práce na cvičeních)

Zkouška

- povinná písemná a doplňková ústní část
- přihlašování prostřednictvím SISu
- k účasti na zkoušce není nutné předchozí získání započtu

Algoritmus

Intuitivní pojem, v literatuře mnoho různých definic, například:

„Konečná posloupnost elementárních příkazů, jejichž provádění umožňuje pro každá přípustná vstupní data mechanickým způsobem získat po konečném počtu kroků příslušná výstupní data.“

Typické vlastnosti (nemusí platit vždy):

- konečnost
- hromadnost (univerzálnost)
- resultativnost
- determinismus

Formální modely algoritmu

rekurzivní funkce (Kurt Gödel, 1934)

Turingův stroj (Alan Turing, 1936)

lambda kalkul (Alonzo Church, 1941)

RAM počítač (Random access machine)

Churchova-Turingova teze:

Ke každému algoritmu existuje ekvivalentní Turingův stroj.

Popis a zápis algoritmu

slovní popis v přirozeném jazyce

pseudokód

program (zjednodušené konstrukce programovacího jazyka)

Největší společný dělitel

X, Y – dvě kladná celá čísla

→ určit největší společný dělitel $\text{NSD}(X, Y)$

Algoritmy:

1. $\text{NSD}(X, Y)$ = největší z celých čísel od 1 do $\min(X, Y)$, které je dělitelem obou čísel X a Y

– postupně zkoušet, nejlépe v pořadí od $\min(X, Y)$ dolů k 1, do nalezení prvního takového společného dělitele

2. Nalézt prvočíselné rozklady čísel X a Y
– jejich maximální společná část určuje $\text{NSD}(X, Y)$

Příklad:

$$396 = \underline{2.2.3.3}.11, \quad 324 = \underline{2.2.3.3}.3.3$$

$$\rightarrow \text{NSD}(396, 324) = 2.2.3.3 = 36$$

3. Eukleidův algoritmus

Eukleidés: Základy (řecky Stoicheia, 13 knih), cca 300 př.n.l.

Idea:

když $X < Y$	$\text{NSD}(X, Y) = \text{NSD}(X, Y-X)$
když $X > Y$	$\text{NSD}(X, Y) = \text{NSD}(X-Y, Y)$
když $X = Y$	$\text{NSD}(X, Y) = X$

Příklad:

$\text{NSD}(396, 324) =$
 $\text{NSD}(72, 324) =$
 $\text{NSD}(72, 252) =$
 $\text{NSD}(72, 180) =$
 $\text{NSD}(72, 108) =$
 $\text{NSD}(72, 36) =$
 $\text{NSD}(36, 36) = 36$

Algoritmus (pro kladná celá čísla X , Y):

dokud $X \neq Y$ dělej

od většího z čísel X , Y odečti menší z čísel X , Y

```
while x != y:  
    if x > y:  
        x -= y  
    else:  
        y -= x  
print(x)
```

Možnost urychlení (pro některé vstupní hodnoty):
místo odčítání použít zbytek po celočíselném dělení

```
while y > 0:  
    z = x % y  
    x = y  
    y = z  
print(x)
```

Program funguje i v případě $X < Y$, jenom vykoná o jednu iteraci více a při první iteraci se hodnoty X , Y navzájem prohodí.

```
while y > 0:  
    x, y = y, x % y  
print(x)
```

Správnost Eukleidova algoritmu

1. Konečnost

= výpočet pro jakákoliv vstupní data skončí

- na začátku výpočtu i stále v jeho průběhu je $X > 0$, $Y > 0$
- v každém kroku výpočtu se hodnota $X+Y$ sníží alespoň o 1
→ nejpozději po $X+Y$ krocích výpočet skončí, je tedy konečný

2. *Parciální (částečná) správnost*

= když výpočet skončí, vydá správný výsledek

- pro $X = Y$ zjevně platí $\text{NSD}(X, Y) = X$
- ukážeme, že pro $X > Y$ platí $\text{NSD}(X, Y) = \text{NSD}(X-Y, Y)$

Důkaz sporem:

Nechť $N = \text{NSD}(X, Y)$, tedy N dělí X a zároveň N dělí Y .

Proto také N dělí $X-Y$ a je tedy N společným dělitelem $X-Y$ a Y .

Pokud by neplatilo, že $N = \text{NSD}(X-Y, Y)$, musí existovat $A > 1$ tak, že $N.A = \text{NSD}(X-Y, Y)$.

Tedy $N.A$ dělí $X-Y$ i Y , takže $N.A$ dělí i jejich součet, což je X .

Jelikož $N.A$ dělí Y a zároveň $N.A$ dělí X , je $N.A$ společným dělitelem čísel X, Y , což je spor s předpokladem, že $N = \text{NSD}(X, Y)$.

Proto skutečně $N = \text{NSD}(X-Y, Y)$.

Efektivita algoritmu (složitost algoritmu)

- časová

počet vykonaných operací (*kterých?*)

→ rychlost výpočtu programu

- prostorová (paměťová)

velikost datových struktur využívaných algoritmem

→ paměť potřebná na uložení dat při výpočtu programu

Kritéria pro hodnocení kvality algoritmu a programu
(příp. praktické použitelnosti programu).

Obě kritéria mohou mířit proti sobě (nelze najednou optimalizovat paměť i čas), musíme zvolit, čemu dáme přednost (dnes obvykle čas).

„výměna času za paměť“

Funkce vyjadřující počet vykonaných operací (resp. velikost potřebné paměti) v závislosti na velikosti vstupních dat. Jako velikost vstupních dat obvykle stačí uvažovat např. počet zpracovaných čísel, nikoliv konkrétní hodnoty (jedná-li se o hodnoty „standardní“ velikosti). Obecně by se musela uvažovat délka zápisu vstupních dat v bitech.

Funkce časové a prostorové složitosti jsou většinou **rostoucí** (nad většími daty bývá výpočet delší).

Jaké operace započítat:

- elementární, vyžadující konstantní čas
- aritmetické, logické, přiřazení
- typické pro řešený problém (převažující)

Kterou paměť započítat:

- do prostorové složitosti nepočítáme paměť potřebnou na uložení vstupních dat, pokud z ní data pouze čteme (neměníme její obsah)

Přesné vyjádření funkce časové složitosti (např. $3.N^2 + 2.N - 4$) je jednak obtížné, jednak většinou zbytečné.

Podstatná je řádová rychlost růstu této funkce pro rostoucí N .

Zanedbáme pomaleji rostoucí členy a konstanty

→ **asymptotická časová složitost** – zapisujeme např. $O(N^2)$

Symbol „velké O“

$f, g: \mathbf{N} \rightarrow \mathbf{R}$

$f \in O(g) \Leftrightarrow \exists c > 0 \exists n_0 > 0 \forall n \geq n_0 : 0 \leq f(n) \leq c.g(n)$

tzn. funkce f se dá **shora** odhadnout funkcí g

(až na multiplikativní konstantu a pro dostatečně velká n)

Opačný odhad **zdola**: $f \in \Omega(g) \quad 0 \leq c.g(n) \leq f(n)$

Přesný (těsný) odhad: $f \in \Theta(g) \Leftrightarrow f \in O(g) \ \& \ f \in \Omega(g)$

Poznámka:

Funkce $3.N^2 + 2.N - 4$ patří nejen do třídy $O(N^2)$, ale podle definice také třeba do třídy $O(N^5)$. Odhad složitosti $O(N^5)$ je zde sice správný, ale zbytečně hrubý a nepřesný, vždy se snažíme o co nejlepší (nejnižší) horní odhad složitosti.

Proto je pro nás cennější, ale také obtížnější, odvodit těsný odhad asymptotické časové složitosti algoritmu $\Theta(N^2)$ než jenom horní odhad $O(N^2)$.

Asymptotická časová složitost

Typické třídy asymptotické časové složitosti algoritmů:

$O(1)$, $O(\log N)$, $O(N)$, $O(N \cdot \log N)$, $O(N^2)$, $O(N^3)$, ..., $O(2^N)$, $O(N!)$

Hledáme algoritmus s co nejmenší asymptotickou časovou složitostí, tedy co nejrychlejší pro velká N . Pro malá N může být třeba i pomalejší než nějaký jiný algoritmus, ale pro malá N to nevadí, doba výpočtu je vždy dostatečně krátká.

- polynomiálně omezený čas – obvykle zvládnutelné i pro velká N
- exponenciální čas – pro větší N časově nezvládnutelné,
použitelné jen v případě, že vstupní data budou vždy malá

Exponenciální časová složitost

Příklad: Ke zpracování vstupních dat velikosti N algoritmus vykoná 2^N operací. Uvažujme rychlost počítače 10^9 operací za sekundu (řádově GHz – dnešní PC).

N	doba výpočtu
20	1 ms
30	1 s
40	17 min
50	11 dní
60	31 let
70	$3 \cdot 10^4$ let
80	$3 \cdot 10^7$ let
90	$3 \cdot 10^{10}$ let
100	$3 \cdot 10^{13}$ let

Pro vyšší hodnoty N (cca 50) je algoritmus prakticky nepoužitelný.
Nepomůže nám ani rychlejší počítač!

Složitost algoritmu v různých případech

Pro každá vstupní data velikosti N nemusí trvat výpočet stejně dlouho, rozdíl může být jen v konstantě, nebo i v asymptotické složitosti.

Časová složitost algoritmu v nejhorším případě

= maximální počet operací vykonaných algoritmem pro nějaká data velikosti N

→ nejčastěji používaná pro hodnocení algoritmů

Časová složitost algoritmu v nejlepším případě

= minimální počet operací vykonaných algoritmem pro nějaká data velikosti N

→ prakticky se nepoužívá

Časová složitost algoritmu v průměrném případě

= průměrný (očekávaný) počet operací vykonaných algoritmem pro data velikosti N (průměr pro všechna možná vstupní data velikosti N)

→ dobře charakterizuje kvalitu algoritmu, ale je obtížné odvodit ji

Složitost problému

- složitost nejlepšího algoritmu (z hlediska časové složitosti), kterým lze řešit daný problém

Nelze odvodit ze složitosti nějakého konkrétního algoritmu ani třeba všech běžně známých algoritmů, charakterizuje problém obecně (jaký nejlepší algoritmus řešící problém může v principu existovat)

→ odvození bývá často obtížné, pro řadu problémů neznáme

Dokázat, že problém má složitost $\Theta(N^2)$ pro nejhorší případ, znamená ukázat, že

- existuje algoritmus, který vyřeší problém s asymptotickou časovou složitostí $O(N^2)$ v nejhorším případě

to bývá snadné – předvedeme jeden zvolený konkrétní algoritmus

- každý algoritmus, který řeší uvedený problém, pracuje v čase $\Omega(N^2)$ v nejhorším případě

to bývá obtížné – nemůžeme ukázat na jednom konkrétním algoritmu

Příklady:

1. Nalézt maximum z daných N čísel $\rightarrow$ časová složitost problému $\Theta(N)$
 - existuje algoritmus s časovou složitostí $O(N)$... triviální
 - nemůže existovat algoritmus s lepší časovou složitostí
 - ... maximem může být kterékoliv z N čísel, takže na každé se musí algoritmus podívat

2. Seřadit daných N čísel podle velikosti (problém třídění)
 - $\rightarrow$ časová složitost problému $\Theta(N \cdot \log N)$
 - existuje algoritmus s časovou složitostí $O(N \cdot \log N)$... např. heapsort
 - nemůže existovat algoritmus s lepší časovou složitostí
 - ... oboje si ukážeme později